

# Afficher une table Supabase en React

R2.09 · Projet KanbanRT · BUT Réseaux & Télécommunications

Ce tutoriel en 6 étapes explique comment charger et afficher les données d'une table Supabase dans un composant React, en utilisant `useState` et `useEffect`.

Étape 1 / 6

## La table Supabase

Votre point de départ : une table dans Supabase. Ici la table `tasks` avec ses colonnes essentielles.

| id (uuid) | title (text)        | status (text) | created_at |
|-----------|---------------------|---------------|------------|
| a1b2...   | Configurer Supabase | done          | 2025-06-01 |
| c3d4...   | Créer TaskCard      | in_progress   | 2025-06-02 |
| e5f6...   | Déployer sur Vercel | todo          | 2025-06-03 |

### Objectif

Afficher ces 3 lignes dans un composant React, en les chargeant dynamiquement depuis Supabase au montage du composant.

Étape 2 / 6

## Étape 1 — Déclarer les états avec `useState`

Deux états suffisent : un tableau pour stocker les données, et un booléen pour afficher un indicateur de chargement pendant la requête.

`src/components/TaskList.jsx`

```
import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';

export default function TaskList() {
  // tasks = tableau vide au départ
  const [tasks, setTasks] = useState([]);
  // loading = true tant que Supabase n'a pas répondu
  const [loading, setLoading] = useState(true);
  // ... suite à l'étape suivante
}
```

### useState([]) vs useState(null)

On initialise avec [] (tableau vide), pas null. Ainsi le .map() dans le JSX fonctionne sans planter avant la fin du chargement.

Étape 3 / 6

## Étape 2 — Charger les données avec useEffect

useEffect s'exécute après le premier rendu. On y place la fonction async qui interroge Supabase. Le tableau vide [] en second argument garantit qu'elle ne s'exécute qu'une seule fois.

Ajoutez ce bloc dans TaskList()

```
useEffect(() => {
  async function fetchTasks() {
    const { data, error } = await supabase
      .from('tasks') // nom de la table
      .select('*') // toutes les colonnes
      .order('created_at', { ascending: false });
    if (error) {
      console.error('Erreur Supabase :', error.message);
    } else {
      setTasks(data); // data est toujours un tableau
    }
    setLoading(false); // fin du chargement dans tous les cas
  }
  fetchTasks();
}, []); // [] = exécuté une seule fois au montage
```

### **Pourquoi définir `fetchTasks` à l'intérieur de `useEffect` ?**

La fonction est async. On ne peut pas passer directement une fonction async à `useEffect` — on la déclare à l'intérieur et on l'appelle aussitôt.

## Étape 3 — Afficher les données dans le JSX

On teste d'abord le chargement, puis on mappe sur le tableau `tasks` pour générer un élément par ligne. Chaque élément doit avoir un attribut `key` unique.

Le `return()` de `TaskList`

```
if (loading) return <p>Chargement...</p>;
return (
  <ul>
    {tasks.map(task => (
      <li
        key={task.id} // obligatoire en React !
        style={{ listStyle: 'none', padding: '8px 0' }}
      >
        <strong>{task.title}</strong>
        <span> – {task.status}</span>
      </li>
    ))}
  </ul>
);
```

### Résultat affiché

Une liste avec autant d'éléments que de lignes dans la table. Si Supabase renvoie 3 tâches, React affiche 3 .

## Code complet — TaskList.jsx

Voici le composant entier, prêt à copier-coller dans votre projet.

`src/components/TaskList.jsx` — version complète

```

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';

export default function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchTasks() {
      const { data, error } = await supabase
        .from('tasks')
        .select('*')
        .order('created_at', { ascending: false });
      if (!error) setTasks(data);
      setLoading(false);
    }
    fetchTasks();
  }, []);

  if (loading) return <p>Chargement...</p>;
  if (tasks.length === 0) return <p>Aucune tâche.</p>;

  return (
    <ul>
      {tasks.map(task => (
        <li key={task.id}>
          {task.title} – {task.status}
        </li>
      ))}
    </ul>
  );
}

```

## Variantes utiles du .select()

La méthode `.select()` accepte plusieurs options pour affiner ce que Supabase renvoie.

```
// Seulement certaines colonnes
.select('id, title, status')

// Avec jointure FK : récupère aussi la catégorie liée
.select('*', categories('*'))

// Filtrer par valeur de colonne
.select('*').eq('status', 'todo')

// Filtrer par utilisateur connecté
.select('*').eq('created_by', user.id)

// Limiter le nombre de résultats
.select('*').limit(10)

// Trier par plusieurs colonnes
.select('*')
.order('priority', { ascending: false })
.order('created_at', { ascending: false })
```

### Récapitulatif des méthodes de filtrage

| Méthode                           | Exemple                                                 | Effet                           |
|-----------------------------------|---------------------------------------------------------|---------------------------------|
| <code>.select(colonnes)</code>    | <code>.select('id, title')</code>                       | Choisir les colonnes retournées |
| <code>.eq(col, val)</code>        | <code>.eq('status', 'todo')</code>                      | Égalité stricte sur une colonne |
| <code>.neq(col, val)</code>       | <code>.neq('status', 'done')</code>                     | Différent de la valeur          |
| <code>.order(col, opts)</code>    | <code>.order('created_at', { ascending: false })</code> | Trier les résultats             |
| <code>.limit(n)</code>            | <code>.limit(5)</code>                                  | Limiter le nombre de lignes     |
| <code>.select('*', T('*'))</code> | <code>.select('*', categories('*'))</code>              | Jointure via clé étrangère      |

### Tableau vide sans erreur ?

Si `data = []` mais `error = null`, c'est souvent le RLS (Row Level Security).

Allez dans Supabase → Authentication → Politiques et vérifiez qu'une politique SELECT existe sur votre table.

## Les 3 règles à retenir

|   |                                              |                                                                                                                                |
|---|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>useState([])</code>                    | Toujours initialiser le tableau avec <code>[]</code> (jamais <code>null</code> ) pour éviter les erreurs <code>.map()</code> . |
| 2 | <code>useEffect(() =&gt; { ... }, [])</code> | Le <code>[]</code> vide en second argument = exécuté une seule fois, au montage du composant.                                  |
| 3 | <code>key={item.id}</code>                   | Chaque élément dans un <code>.map()</code> doit avoir un attribut <code>key</code> unique — React l'exige.                     |